INGENIERÍA BACKEND: SERVIDORES, SEGURIDAD Y ARQUITECTURA DEFENSIVA

Desarrollo con Node.js 24 LTS, Express 5, Autenticación Criptográfica JWT, Hashing Bcrypt y OWASP Top 10

Manual técnico, metodológico y de seguridad aplicada para ingenieros backend, arquitectos de software y desarrolladores full-stack. Elaborado bajo los estándares de desarrollo de software industrial de Onetel C.A. Este documento aborda la ingeniería de servidores de producción sobre Node.js: arquitectura en capas desacopladas, ciclo de vida de peticiones HTTP, middlewares perimetrales de seguridad (Helmet, CORS estricto, Rate Limiting), validación exhaustiva de esquemas con Zod, autenticación sin estado (*stateless*) mediante tokens JWT (RFC 7519) con rotación de Refresh Tokens, cifrado de contraseñas con sal dinámica (Bcrypt cost factor = 12), mitigación activa de Timing Attacks y la implementación de una suite automatizada de pruebas y auditoría técnica.

Módulo	Marco Conceptual, Hitos Técnicos y Competencias	Stack y Herramientas
Módulo 1	Arquitectura Interna de Node.js: V8, Libuv, Thread Pool y Modelo No Bloqueante	Node.js 24 LTS / Libuv
Módulo 2	Desarrollo de Servidores Express 5: Ciclo Request-Response y Manejo Asíncrono	Express 5.1+ / ESM
Módulo 3	Arquitectura en Capas Limpia: Rutas, Controladores, Servicios y Repositorios	Clean Layered Architecture
Módulo 4	Validación Estricta de Entorno y Cargas Útiles con Zod (Fail-Fast System)	Zod 3.24+ / Type Inference
Módulo 5	Seguridad Defensiva OWASP: Cabeceras Helmet, Control CORS y Límite de Payload	Helmet 8 / CORS / Anti-DoS
Módulo 6	Protección Perimetral contra Fuerza Bruta: Rate Limiting por IP y Endpoint	Express Rate Limit / DDoS
Módulo 7	Criptografía de Contraseñas: Algoritmo Bcrypt, Salt Aleatoria y Cost Factor	Bcryptjs / Salting
Módulo 8	Vulnerabilidad de Timing Attacks en Login y Mitigación con Dummy Hash	Constant-Time Verification
Módulo 9	Autenticación JWT RFC 7519: Access Tokens de Vida Corta y Firma Criptográfica	JsonWebToken / HS256
Módulo 10	Arquitectura de Sesión Segura: Refresh Tokens, Rotación y Lista Negra	Token Revocation / DB Blacklist
Módulo 11	Middlewares de Autorización, RBAC y Manejo Centralizado de Excepciones	Error Middleware / RBAC
Prácticas	Laboratorios Guiados: Auth API, Pruebas de Fuerza Bruta y Checklist Técnico	Hands-On Lab / 20 Criterios

Nombre del Ingeniero:

Empresa / Proyecto:

Cédula / ID Profesional:

Instructor Responsable:

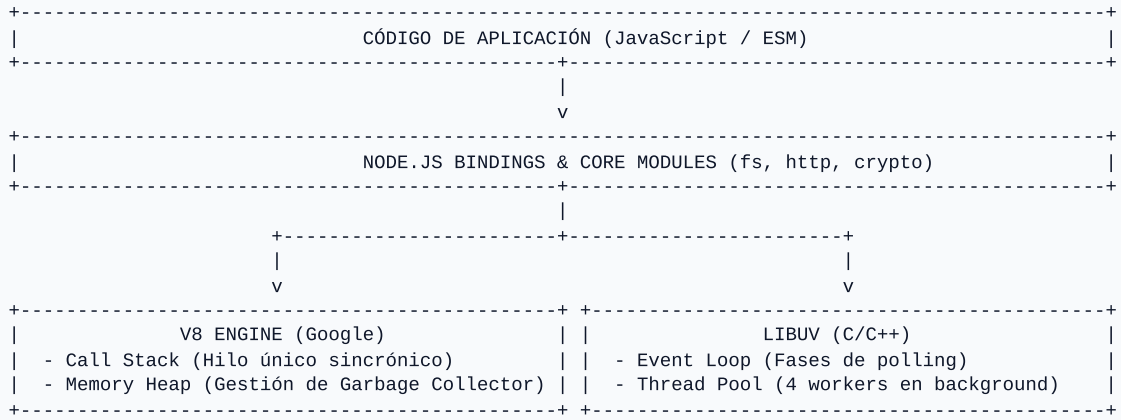
1.1 EL MOTOR V8, LIBUV Y EL MODELO DE I/O ASÍNCRONO NO BLOQUEANTE

NODE.JS INTERNALS

¿Cómo Procesa Node.js Concurrencia Masiva sobre un Solo Hilo?

La Dualidad V8 + Libuv: Node.js no ejecuta código multitarea de forma tradicional (un hilo por conexión como Apache o Tomcat). Se compone de dos motores:
• **Google V8:** El motor open-source que compila y ejecuta JavaScript directamente a código máquina nativo sobre el *Main Thread*.
• **Libuv (C/C++):** Librería multiplataforma que abstrae las operaciones asíncronas del sistema operativo (epoll en Linux, kqueue en macOS, IOCP en Windows) y mantiene un **Thread Pool interno de 4 hilos** (configurable con UV_THREADPOOL_SIZE) para tareas costosas que no pueden resolverse de forma no bloqueante a nivel de kernel: operaciones de disco (`fs`), resolución DNS y funciones criptográficas pesadas (`crypto.pbkdf2`, compresión zlib).

DIAGRAMA: ARQUITECTURA DE EJECUCIÓN EN NODE.JS — Modelo de Capas



Novedades Críticas de Node.js 24 LTS para Servidores de Producción

Característica	Comportamiento en Node.js 24 LTS	Beneficio de Producción
Carga Nativa de Entorno	`node --env-file=.env src/server.js` nativo sin librerías externas.	Cero dependencias npm para lectura segura de variables iniciales.
Modo Watch de Desarrollo	`node --watch src/server.js` reinicia el proceso ante cambios de código.	Elimina la necesidad de instalar herramientas pesadas como nodemon.
Módulo de Pruebas Nativo	`node:test` y `node:assert/strict` incluidos en el core.	Suites de testing unitario sin frameworks externos de terceros.
Fetch API y Web Streams	Soporte completo y maduro para `fetch`, `Headers` y `FormData`.	Intercambio de código cliente-servidor 100% estandarizado.

2.1 EXPRESS 5.1+: EL SALTO HISTÓRICO EN MANEJO DE PROMESAS

EXPRESS 5 ARCHITECTURE

La Revolución de Express 5 frente a Express 4

La Caída Silenciosa de Servidores en Express 4: En Express 4, si una función de ruta declarada con `async (req, res) => { ... }` lanzaba una excepción no capturada (ej. fallo de conexión a base de datos), la promesa rechazada **se perdía en el Event Loop**, provocando un evento `unhandledRejection` y congelando la conexión del cliente hasta el timeout de red, o tirando el proceso de Node.
Express 5.1+ reescribió el enrutador interno: ahora **intercepta automáticamente las promesas rechazadas** en controladores asíncronos y las envía de forma transparente al middleware centralizado de errores con firma `(err, req, res, next)`, erradicando la necesidad de envolver cada controlador en bloques `try/catch` manuales o librerías puente como `express-async-errors`.

JAVASCRIPT: SERVIDOR BASE EXPRESS 5 MODULAR EN ES MODULES — src/app.js

```javascript
import express from 'express';

export function crearServidor() {
  const app = express();

  // 1. Deserializador seguro de JSON con límite estricto de tamaño
  app.use(express.json({ limit: '10kb' }));

  // 2. Endpoint de salud para balanceadores de carga y sondas Kubernetes/Docker
  app.get('/health', (req, res) => {
    res.status(200).json({
      status: 'healthy',
      uptime: process.uptime(),
      timestamp: new Date().toISOString(),
      memoria: process.memoryUsage()
    });
  });

  // 3. Ruta de demostración de captura asíncrona nativa de Express 5 (SIN try/catch)
  app.get('/api/v1/prueba-error', async (req, res) => {
    // Simulamos una excepción asíncrona en base de datos
    throw new Error("Fallo simulado: Base de datos no disponible");
    // Express 5 intercepta la promesa rechazada y salta directamente al middleware de error
  });

  // 4. Manejo de rutas inexistentes (404 Not Found)
  app.use((req, res) => {
    res.status(404).json({ error: `La ruta ${req.method} ${req.url} no existe en este servidor.` });
  });

  // 5. Middleware Centralizado de Excepciones (Obligatorio 4 parámetros)
  app.use((err, req, res, next) => {
    console.error(`[ERROR CAPTURADO]: ${err.message}`);
    res.status(err.status || 500).json({
      error: err.message || "Error interno del servidor",
      timestamp: new Date().toISOString()
    });
  });

  return app;
}
```

Firma Estricta de 4 Argumentos: Express determina que una función es un middleware de captura de errores evaluando `fn.length === 4`. Si omites el parámetro `next` aunque no lo uses, Express lo tratará como una ruta ordinaria y la excepción no será capturada.

3.1 ARQUITECTURA EN CAPAS: ROUTES, CONTROLLERS, SERVICES Y REPOSITORIES

CLEAN ARCHITECTURE

Separación Estricta de Responsabilidades (SoC)

El Patrón Antidiseño "Controlador Monolítico": Escribir consultas SQL, reglas de validación, lógica de negocio y parseo HTTP dentro del callback de una ruta genera código imposible de testear y propenso a fallos. La ingeniería limpia desacopla el flujo en 4 capas unidireccionales:
• **1. Routes (Enrutamiento):** Define verbos HTTP y endpoints. Aplica middlewares de seguridad y autorización.
• **2. Controllers (Controladores):** Extrae parámetros de `req.body` o `req.params`, invoca a los servicios y envía la respuesta HTTP adecuada.
• **3. Services (Servicios de Dominio):** Contiene la lógica de negocio pura agnóstica del protocolo HTTP (cálculos, orquestación, reglas de validación de negocio).
• **4. Repositories / Data Access (Persistencia):** Interactúa directamente con la base de datos (PostgreSQL, SQLite, Redis).

TEXT: ÁRBOL DE DIRECTORIOS EMPRESARIAL DE SERVICIO BACKEND — Estructura src/

```
backend-onetel-core/
├── package.json
├── .env.example
├── src/
│   ├── config/          # Variables de entorno y configuraciones de conexión
│   ├── routes/          # Definición de endpoints HTTP
│   ├── controllers/     # Orquestación de peticiones y respuestas
│   ├── services/        # Lógica de negocio y casos de uso
│   ├── repositories/    # Consultas a bases de datos y persistencia
│   ├── middlewares/     # Interceptores de seguridad, validación y auth
│   ├── schemas/         # Esquemas de validación Zod de entradas
│   ├── utils/           # Claves criptográficas, helpers y formateadores
│   ├── app.js           # Ensamblaje de Express sin escuchar puerto (Testable)
│   └── server.js        # Punto de entrada que levanta el puerto HTTP
```

JAVASCRIPT: FLUJO CONTROLADOR-SERVICIO LIMPIO — src/services/servidor.service.js

```javascript
// Capa de Servicio: Lógica de negocio pura (fácil de testear con Mocks)
export class ServidorService {
  constructor(servidorRepository) {
    this.repository = servidorRepository;
  }

  async registrarNuevoNodo(datosValidados) {
    const existe = await this.repository.buscarPorHostname(datosValidados.hostname);
    if (existe) {
      const error = new Error("El hostname ya está en uso por otro nodo del cluster.");
      error.status = 409; // Conflict
      throw error;
    }
    return await this.repository.crear(datosValidados);
  }
}
```

4.1 VALIDACIÓN DE ESQUEMAS: ENTORNO Y CARGAS ÚTILES CON ZOD

SCHEMA VALIDATION

El Principio Fail-Fast (Falla Rápido, Falla Seguro)

Blindaje de Entorno al Arrancar: Un servidor jamás debe encender si falta una variable crítica de configuración (ej. JWT_SECRET no definida o con menos de 32 caracteres). Si el servidor arranca y el primer usuario intenta loguearse horas después, ocurrirá una excepción catastrófica en runtime. Mediante **Zod**, definimos esquemas de tipado estricto que abortan el proceso inmediatamente al encender si las variables no son válidas.

JAVASCRIPT: ESQUEMA DE ENTORNO BLINDADO (src/config/env.js) — src/config/env.js

```
import { z } from 'zod';

const envSchema = z.object({
  PORT: z.coerce.number().min(1024).max(65535).default(3000),
  NODE_ENV: z.enum(['development', 'production', 'test']).default('development'),
  JWT_SECRET: z.string().min(32, "JWT_SECRET requiere un mínimo de 32 caracteres criptográficos"),
  JWT_EXPIRES_IN: z.string().default("15m"),
  REFRESH_SECRET: z.string().min(32, "REFRESH_SECRET requiere al menos 32 caracteres"),
  REFRESH_EXPIRES_IN: z.string().default("7d"),
  CORS_ORIGIN: z.string().default("http://localhost:5173"),
});

// Validación síncrona en el momento de la carga
const resultado = envSchema.safeParse(process.env);

if (!resultado.success) {
  console.error("❌ ERROR CRÍTICO DE ENTORNO: La configuración de arranque es inválida:");
  console.error(JSON.stringify(resultado.error.format(), null, 2));
  process.exit(1); // Detención forzada inmediata
}

export const env = resultado.data;
```

4.2 Middleware Universal de Validación de Peticiones

JAVASCRIPT: INTERCEPTOR DE ESQUEMAS ZOD PARA BODY/PARAMS/QUERY — src/middlewares/validate.js

```
export const validar = (esquema) => async (req, res, next) => {
  const resultado = await esquema.safeParseAsync({
    body: req.body,
    query: req.query,
    params: req.params,
  });

  if (!resultado.success) {
    return res.status(422).json({
      error: "Validación de datos fallida",
      detalles: resultado.error.issues.map(i => ({ campo: i.path.join('.'), mensaje: i.message }))
    });
  }

  req.datosValidados = resultado.data;
  next();
};
```

5.1 PROTECCIÓN DE CABECERAS HTTP (HELMET) Y POLÍTICAS CORS ESTRICTAS

PERIMETER SECURITY

Blindaje Contra Ataques Comunes de OWASP con Helmet

Ocultamiento de Huella Tecnológica: Por defecto, Express emite la cabecera `X-Powered-By: Express`, informando a atacantes y escáneres de vulnerabilidades la tecnología exacta del servidor. **Helmet** es una suite que configura 15 middlewares de cabeceras HTTP de seguridad para mitigar ataques como Clickjacking, ataques de inyección MIME sniffing y Cross-Site Scripting (XSS).

Cabecera HTTP de Seguridad	Configuración Helmet	Ataque que Neutraliza
`X-Powered-By`	`hidePoweredBy()`	Elimina la cabecera que expone la versión del framework a atacantes.
`X-Frame-Options`	`frameguard({ action: 'deny' })`	Impide que el sitio sea embebido en un `<iframe>` (Clickjacking).
`X-Content-Type-Options`	`noSniff()`	Fuerza al navegador a respetar el MIME-type (evita ejecución de scripts camuflados).
`Content-Security-Policy`	`contentSecurityPolicy()`	Restringe dominios autorizados de scripts, fuentes e imágenes (Anti-XSS).

5.2 Configuración Estricta de Cross-Origin Resource Sharing (CORS)

JAVASCRIPT: SUITE PERIMETRAL HELMET Y POLÍTICA CORS (src/middlewares/security.js) — src/middlewares/security.js

```
import helmet from 'helmet';
import cors from 'cors';
import { env } from '../config/env.js';

export function configurarSeguridadPerimetral(app) {
  // 1. Blindaje global de cabeceras HTTP
  app.use(helmet({
    contentSecurityPolicy: env.NODE_ENV === 'production',
    crossOriginEmbedderPolicy: true,
  }));

  // 2. Política CORS con lista blanca explícita (NUNCA usar origin: '*' con credenciales)
  app.use(cors({
    origin: (origin, callback) => {
      const origenesPermitidos = [env.CORS_ORIGIN, 'http://localhost:5173'];
      if (!origin || origenesPermitidos.includes(origin)) {
        callback(null, true);
      } else {
        callback(new Error('Bloqueado por política de seguridad CORS Onetel'));
      }
    },
    methods: ['GET', 'POST', 'PUT', 'DELETE'],
    allowedHeaders: ['Content-Type', 'Authorization'],
    credentials: true,
    maxAge: 86400 // Cache de pre-vuelo (Preflight OPTIONS) por 24 horas
  }));
}
```

6.1 MITIGACIÓN DE ATAQUES DE DICCIONARIO Y DOS CON RATE LIMITING

RATE LIMITING DEFENSE

Mecánica de Limitación de Solicitudes (Leaky Bucket / Fixed Window)

Ataques de Denegación de Servicio y Fuerza Bruta: Si un endpoint de inicio de sesión no limita la tasa de peticiones, un atacante con un script en Python o Go puede probar 10,000 contraseñas por minuto. **express-rate-limit** implementa una ventana de tiempo en memoria (o en Redis para clusters) que rastrea las solicitudes por dirección IP. Si se superan los umbrales autorizados, el servidor responde inmediatamente con código `429 Too Many Requests` y las cabeceras estándar RFC `Retry-After` y `RateLimit-Reset`.

JAVASCRIPT: CONFIGURACIÓN DE RATE LIMITERS POR SENSIBILIDAD — src/middlewares/rateLimit.js

```
import rateLimit from 'express-rate-limit';

// 1. Limitador Perimetral Global (Aplica a todas las rutas del API)
export const limitadorGlobal = rateLimit({
  windowMs: 15 * 60 * 1000, // Ventana de 15 minutos
  max: 100, // Límite de 100 solicitudes por ventana por IP
  standardHeaders: true, // Retorna cabeceras RFC 'RateLimit-*'
  legacyHeaders: false,  // Desactiva cabeceras antiguas 'X-RateLimit-*'
  message: {
    error: "Límite de peticiones alcanzado. Por favor intente más tarde.",
    statusCode: 429
  }
});

// 2. Limitador Estricto para Rutas Sensibles (Login / Registro / Reset Password)
export const limitadorAuthEstricto = rateLimit({
  windowMs: 15 * 60 * 1000, // Ventana de 15 minutos
  max: 5, // Máximo 5 intentos fallidos permitidos
  skipSuccessfulRequests: true, // Si el usuario ingresa con éxito, no consume cuota
  message: {
    error: "Demasiados intentos fallidos de autenticación. Su acceso temporal ha sido bloqueado por 15 minutos.",
    statusCode: 429
  }
});
```

Parámetro skipSuccessfulRequests: Al configurar `skipSuccessfulRequests: true` en el endpoint de login, un usuario legítimo que inicia sesión varias veces en el día no se verá bloqueado. Únicamente se contabilizan los intentos fallidos con credenciales erróneas.

7.1 CRIPTOGRAFÍA DE CONTRASEÑAS: BCRYPT, SALT DINÁMICA Y WORK FACTOR

PASSWORD CRYPTOGRAPHY

¿Por Qué las Funciones Hash Rápidas Están Prohibidas para Contraseñas?

La Trampa de SHA-256 o MD5: Algoritmos como SHA-256 o MD5 están diseñados para verificar la integridad de archivos a máxima velocidad. Una tarjeta gráfica moderna (GPU) puede calcular más de 5,000,000,000 de hashes SHA-256 por segundo. Si una base de datos con contraseñas en SHA-256 es filtrada, un atacante descifrará el 90% de las contraseñas en minutos usando *Rainbow Tables*. **Bcrypt** es una función lenta por diseño (basada en el cifrado Blowfish). Su costo computacional es exponencial (2^{cost}). Configurado en `cost = 12`, requiere ~150 ms por hash en un procesador moderno, imposibilitando ataques de fuerza bruta masivos. Además, incorpora una **sal aleatoria (salt) de 128 bits** en cada hash, garantizando que dos usuarios con la misma contraseña exacta generen hashes completamente distintos.

JAVASCRIPT: MÓDULO DE HASHING ASÍNCRONO ROBUSTO — src/utils/hash.js

```javascript
import bcrypt from 'bcryptjs';

// Cost Factor 12 = 2^12 = 4,096 rondas de cifrado (~120-150ms de cómputo)
const COST_FACTOR = 12;

/**
 * Genera un hash unidireccional seguro con sal aleatoria
 */
export async function hashearPassword(passwordPlano) {
  // bcrypt.hash() genera la salt aleatoria y el hash en una sola operación atómica
  return await bcrypt.hash(passwordPlano, COST_FACTOR);
}

/**
 * Compara una contraseña plana contra el hash almacenado
 */
export async function verificarPassword(passwordPlano, hashAlmacenado) {
  return await bcrypt.compare(passwordPlano, hashAlmacenado);
}
```

Anatomía del String Bcrypt: Un hash de Bcrypt tiene exactamente 60 caracteres y la forma `$2a$12$e8k6mN1rN8P5E1g0oE7cEuZp4iN9fS5kOG0mN1rN8P5E1g0oE7cEu`. Donde `$2a$` indica la versión del algoritmo, `$12$` es el factor de costo, los siguientes 22 caracteres son la salt aleatoria, y los 31 finales son el hash sellado.

8.1 ATAQUES DE CANAL LATERAL (TIMING ATTACKS) Y DEFENSA CONSTANT-TIME

SIDE-CHANNEL DEFENSE

¿Cómo un Atacante Descubre Correos Válidos Mediante Milisegundos?

Vulnerabilidad de Enumeración de Usuarios por Tiempo: Considera el flujo tradicional de login:
`const user = await db.find({ email }); if (!user) return res.status(401);`
Si el usuario no existe, la base de datos responde en 2 ms y el endpoint retorna inmediatamente. Si el usuario existe, el servidor ejecuta `bcrypt.compare()`, tardando 140 ms adicionales. Un atacante midiendo los tiempos de respuesta HTTP con un script automatizado sabe con 100% de certeza qué correos electrónicos están registrados en la empresa simplemente observando la latencia.
Defensa Defensiva Onetel: Si el usuario no existe en la base de datos, ejecutamos deliberadamente `bcrypt.compare()` contra un **hash dummy precalculado legítimo** de 60 caracteres. El tiempo de respuesta será constante (~140 ms) independientemente de si el usuario existe o no.

JAVASCRIPT: CONTROLADOR DE LOGIN INMUNE A TIMING ATTACKS — src/controllers/auth.controller.js

```javascript
import { verificarPassword } from '../utils/hash.js';

// Hash dummy válido de 60 caracteres precalculado con cost = 12
const DUMMY_HASH = '$2a$12$e8k6mN1rN8P5E1g0oE7cEuZp4iN9fS5k0G0mN1rN8P5E1g0oE7cEu';

export async function loginSeguro(req, res) {
  const { email, password } = req.body;

  // 1. Búsqueda de usuario en la base de datos
  const usuario = await baseDatos.buscarPorEmail(email);

  // 2. Selección del hash objetivo (Target Hash):
  // Si el usuario existe, usamos su hash real. Si NO existe, usamos el DUMMY_HASH.
  const hashAComparar = usuario ? usuario.passwordHash : DUMMY_HASH;

  // 3. La verificación computacional SIEMPRE se ejecuta consumiendo ~140ms
  const esValida = await verificarPassword(password, hashAComparar);

  // 4. Fallo unificado: Mismo mensaje y mismo tiempo para correo o clave inválida
  if (!usuario || !esValida) {
    return res.status(401).json({ error: "Credenciales de acceso inválidas." });
  }

  // 5. Emisión de credenciales JWT
  const accessToken = firmarToken({ id: usuario.id, email: usuario.email });
  res.json({ status: 'ok', accessToken });
}
```

Mensajes de Error Genéricos: Jamás respondas "El usuario no existe" o "Contraseña incorrecta". Responde siempre la frase genérica `"Credenciales de acceso inválidas"` tanto para fallos de usuario como de contraseña.

9.1 TOKENS JWT (RFC 7519): FIRMA CRIPTOGRÁFICA Y ACCESS TOKENS

JWT STATELESS AUTH

Anatomía y Mecánica de Verificación de un JWT

Estructura Compacta en Tres Partes: Un JWT consta de `Header.Payload.Signature` codificadas en Base64URL:
• **Header:** Especifica el algoritmo criptográfico (`HS256` para HMAC simétrico o `RS256` para par público/privado).
• **Payload (Claims):** Información no confidencial (`id`, `email`, rol, fecha de expiración `exp`). *¡Nunca coloques contraseñas en el payload!*
• **Signature:** Firma calculada como `HMAC-SHA256(Base64(Header) + "." + Base64(Payload), Secret)`. Si un atacante modifica el ID de usuario en el payload, la firma matemática ya no coincidirá y el servidor rechazará la petición sin tocar la base de datos.

JAVASCRIPT: MÓDULO DE FIRMA Y VERIFICACIÓN CRIPTOGRÁFICA DE TOKENS — src/utils/jwt.js

```
import jwt from 'jsonwebtoken';
import { env } from '../config/env.js';

/**
 * Emite un Access Token de vida corta firmado criptográficamente
 */
export function generarAccessToken(payloadUsuario) {
  return jwt.sign(
    {
      sub: payloadUsuario.id,
      email: payloadUsuario.email,
      rol: payloadUsuario.rol || 'OPERADOR'
    },
    env.JWT_SECRET,
    {
      expiresIn: env.JWT_EXPIRES_IN, // 15 minutos
      algorithm: 'HS256'
    }
  );
}

/**
 * Verifica y descifra la firma criptográfica del token
 */
export function verificarAccessToken(token) {
  return jwt.verify(token, env.JWT_SECRET, { algorithms: ['HS256'] });
}
```

Declaraciones Estándar (Claims) del Payload RFC 7519

Claim	Nombre Completo	Propósito y Significado Técnico
`sub`	Subject	Identificador único e inmutable del usuario (ID de base de datos).
`iat`	Issued At	Marca de tiempo Unix de la fecha/hora en que fue emitido el token.
`exp`	Expiration Time	Marca de tiempo Unix a partir de la cual el token queda inválido.

10.1 SESIÓN SEGURA: ROTACIÓN DE REFRESH TOKENS Y LISTA NEGRA

TOKEN ROTATION & REVOCATION

El Problema de la Inmutabilidad de los JWT

¿Cómo Revocar un Token si es Robado? Como los JWT son sin estado (*stateless*), si un token tiene una vigencia de 30 días y es interceptado, el atacante tendrá acceso durante un mes entero sin que el servidor pueda anularlo.
Estrategia Dual de Dos Tokens (Access + Refresh):
1. **Access Token:** Vida ultracorta (15 minutos). Se envía en cada cabecera HTTP `Authorization: Bearer <token>`.
2. **Refresh Token:** Vida media (7 días). Se almacena de forma segura en base de datos o cookies `HttpOnly`. Solo se utiliza contra el endpoint `/api/auth/refresh` para obtener un nuevo Access Token cuando el anterior expire.
3. **Rotación Obligatoria:** Cada vez que se usa un Refresh Token, el servidor lo revoca, lo destruye y emite uno nuevo. Si un Refresh Token viejo se presenta de nuevo, se detecta un robo de sesión y se invalidan todos los tokens de ese usuario.

JAVASCRIPT: ENDPOINT DE REFRESCO Y REVOCACIÓN DE SESIONES — src/controllers/refresh.controller.js

```
import jwt from 'jsonwebtoken';
import { env } from '../config/env.js';
import { generarAccessToken } from '../utils/jwt.js';

// Lista de revocación de Refresh Tokens (En producción: tabla en PostgreSQL o set en Redis)
const refreshTokensRevocados = new Set();

export async function refrescarSesion(req, res) {
  const { refreshToken } = req.body;
  if (!refreshToken) return res.status(400).json({ error: "Refresh token requerido." });

  // 1. Comprobación de revocación en lista negra
  if (refreshTokensRevocados.has(refreshToken)) {
    return res.status(403).json({ error: "Sesión revocada o comprometida. Inicie sesión de nuevo." });
  }

  try {
    // 2. Verificación criptográfica con la clave de refresh
    const payload = jwt.verify(refreshToken, env.REFRESH_SECRET);

    // 3. Revocar el token usado (Rotación de tokens de un solo uso)
    refreshTokensRevocados.add(refreshToken);

    // 4. Emitir nuevo par de credenciales
    const nuevoAccessToken = generarAccessToken({ id: payload.sub, email: payload.email, rol: payload.rol });
    const nuevoRefreshToken = jwt.sign(
      { sub: payload.sub, email: payload.email, rol: payload.rol },
      env.REFRESH_SECRET,
      { expiresIn: env.REFRESH_EXPIRES_IN }
    );

    res.json({ accessToken: nuevoAccessToken, refreshToken: nuevoRefreshToken });
  } catch (err) {
    return res.status(403).json({ error: "Refresh token expirado o firma inválida." });
  }
}
```

Higiene de Memoria en Redis: Al usar Redis para la lista negra de revocación, se configura un tiempo de expiración TTL idéntico a la vida del Refresh Token (7 días). Pasado ese tiempo, Redis purga la clave automáticamente sin saturar la memoria.

11.1 MIDDLEWARE DE AUTENTICACIÓN Y CONTROL DE ACCESO RBAC

AUTHORIZATION PIPELINE

Autenticación vs. Autorización

Identidad vs. Privilegios:
• **Autenticación (AuthN):** ¿Quién eres? Se valida extrayendo y verificando el JWT de la cabecera `Authorization: Bearer <token>`. Inyecta el usuario decodificado en `req.usuario`.
• **Autorización (AuthZ / RBAC):** ¿Qué tienes permitido hacer? Evalúa si el `req.usuario.rol` tiene los privilegios necesarios para ejecutar una acción destructiva (ej. reiniciar un nodo industrial o borrar un registro).

JAVASCRIPT: MIDDLEWARES DE PROTECCIÓN Y CONTROL RBAC — src/middlewares/auth.js

```
import { verificarAccessToken } from '../utils/jwt.js';

// Middleware de Autenticación de Identidad
export function requerirAutenticacion(req, res, next) {
  const cabecera = req.headers.authorization;
  if (!cabecera || !cabecera.startsWith('Bearer ')) {
    return res.status(401).json({ error: "Acceso denegado: Cabecera Authorization requerida en formato Bearer." });
  }

  const token = cabecera.slice(7).trim();

  try {
    const decoded = verificarAccessToken(token);
    // Inyección de contexto seguro para los siguientes middlewares y controladores
    req.usuario = {
      id: decoded.sub,
      email: decoded.email,
      rol: decoded.rol
    };
    next();
  } catch (err) {
    const mensaje = err.name === 'TokenExpiredError' ? "Token expirado." : "Token inválido o firma alterada.";
    return res.status(401).json({ error: mensaje });
  }
}

// Middleware de Autorización Basada en Roles (RBAC)
export function autorizarRoles(...rolesPermitidos) {
  return (req, res, next) => {
    if (!req.usuario) {
      return res.status(500).json({ error: "Error interno: Middleware RBAC invocado sin autenticación previa." });
    }

    if (!rolesPermitidos.includes(req.usuario.rol)) {
      return res.status(403).json({
        error: `Acceso prohibido: Su rol [${req.usuario.rol}] no posee los privilegios requeridos.`
      });
    }

    next();
  };
}
```

Diferencia HTTP 401 vs 403: Devuelve código `401 Unauthorized` cuando el cliente no presenta credenciales o su token expiró (no sabemos quién es). Devuelve código `403 Forbidden` cuando sabemos perfectamente quién es, pero sus permisos son insuficientes para acceder al recurso.

LABORATORIO 1: ENSAMBLAJE DE LA API RESTFUL DE AUTENTICACIÓN SEGURA

HANDS-ON LAB 1

Objetivo del Laboratorio: Construir el servidor Express 5 de producción integrando Helmet, CORS, Rate Limiting, hash Bcrypt con salt dinámico y endpoints de registro y login blindados contra ataques.

JAVASCRIPT: SERVIDOR DE PRODUCCIÓN COMPLETO (src/server-auth.js) — src/server-auth.js

```
import express from 'express';
import helmet from 'helmet';
import cors from 'cors';
import { limitadorGlobal, limitadorAuthEstricto } from './middlewares/rateLimit.js';
import { hashearPassword, verificarPassword } from './utils/hash.js';
import { generarAccessToken, verificarAccessToken } from './utils/jwt.js';

const app = express();
const DUMMY_HASH = '$2a$12$e8k6mN1rN8P5E1g0oE7cEuZp4iN9fS5k0G0mN1rN8P5E1g0oE7cEu';

// Middlewares de infraestructura y seguridad perimetral
app.use(helmet());
app.use(cors({ origin: 'http://localhost:5173', credentials: true }));
app.use(express.json({ limit: '10kb' }));
app.use(limitadorGlobal);

// Base de datos en memoria para propósitos didácticos
const usuariosDB = [];

// 1. Endpoint de Registro de Usuarios
app.post('/api/auth/register', async (req, res) => {
  const { email, password, rol = 'OPERADOR' } = req.body;
  if (!email || !password || password.length < 8) {
    return res.status(400).json({ error: "Datos inválidos. Password requiere mínimo 8 caracteres." });
  }

  const existe = usuariosDB.find(u => u.email === email);
  if (existe) return res.status(409).json({ error: "El correo ya se encuentra registrado." });

  const hash = await hashearPassword(password);
  const nuevoUsuario = { id: usuariosDB.length + 1, email, passwordHash: hash, rol };
  usuariosDB.push(nuevoUsuario);

  res.status(201).json({ mensaje: "Usuario registrado exitosamente.", id: nuevoUsuario.id });
});

// 2. Endpoint de Login con Mitigación de Timing Attack y Rate Limiting
app.post('/api/auth/login', limitadorAuthEstricto, async (req, res) => {
  const { email, password } = req.body;
  const usuario = usuariosDB.find(u => u.email === email);
  const hashComparar = usuario ? usuario.passwordHash : DUMMY_HASH;
  const esValida = await verificarPassword(password, hashComparar);

  if (!usuario || !esValida) {
    return res.status(401).json({ error: "Credenciales de acceso inválidas." });
  }

  const token = generarAccessToken(usuario);
  res.json({ token, expiraEn: "15m" });
});

// 3. Endpoint Privado Protegido
app.get('/api/servidor/estado', (req, res) => {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith('Bearer ')) return res.status(401).json({ error: "No autorizado" });
  try {
    const decodificado = verificarAccessToken(authHeader.slice(7));
    res.json({ status: 'operacional', consultadoPor: decodificado.email, rol: decodificado.rol });
  } catch {
    res.status(401).json({ error: "Token inválido o vencido" });
  }
});

app.listen(3000, () => console.log("✔ API de Autenticación Segura corriendo en puerto 3000"));
```

LABORATORIO 2: PRUEBAS DE PENETRACIÓN (FUERZA BRUTA Y TIMING AUDIT)

SECURITY TESTING LAB

Objetivo del Laboratorio: Utilizar scripts de terminal en Bash con `curl` para simular un ataque automatizado de fuerza bruta comprobando que el Rate Limiter bloquea al atacante con HTTP 429, y auditar la latencia de respuesta para confirmar la defensa constant-time.

BASH: SCRIPT DE AUDITORÍA DE SEGURIDAD Y FUERZA BRUTA — test-seguridad.sh

```bash
#!/bin/bash
set -e

URL_BASE="http://localhost:3000"
echo "============================================================="
echo "    AUDITORÍA DE SEGURIDAD DEFENSIVA BACKEND — ONETEL C.A.     "
echo "============================================================="

# 1. Registrar un usuario legítimo de prueba
echo "[1/4] Creando usuario legítimo de prueba..."
curl -s -X POST "$URL_BASE/api/auth/register" \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@onetel.cloud","password":"PasswordSeguro123!","rol":"ADMIN"}' | jq .

# 2. Auditar Cabeceras de Seguridad con Helmet (Verificar ausencia de X-Powered-By)
echo "[2/4] Auditando cabeceras de seguridad HTTP..."
CABECERAS=$(curl -s -I "$URL_BASE/health")
if echo "$CABECERAS" | grep -qi "X-Powered-By"; then
  echo "❌ FALLO: La cabecera X-Powered-By está expuesta."
else
  echo "✔ ÉXITO: Helmet ocultó exitosamente la huella tecnológica del servidor."
fi

# 3. Prueba de Carga y Simulación de Ataque de Fuerza Bruta (6 intentos erróneos)
echo "[3/4] Lanzando ráfaga de intentos erróneos de login..."
for i in {1..6}; do
  CODIGO=$(curl -s -o /dev/null -w "%{http_code}" -X POST "$URL_BASE/api/auth/login" \
    -H "Content-Type: application/json" \
    -d '{"email":"admin@onetel.cloud","password":"ClaveIncorrecta"}')
  echo "Intento #$i: Código de respuesta HTTP $CODIGO"
done
# Esperado: Intento 1 a 5 retornan 401. El intento 6 RETORNA 429 (Too Many Requests).

# 4. Prueba de Acceso al Endpoint Protegido con Token Válido
echo "[4/4] Probando acceso autorizado con credenciales válidas..."
# Obtenemos token con un usuario no bloqueado
TOKEN=$(curl -s -X POST "$URL_BASE/api/auth/login" \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@onetel.cloud","password":"PasswordSeguro123!"}' | jq -r .token)

if [ "$TOKEN" != "null" ] && [ -n "$TOKEN" ]; then
  curl -s "$URL_BASE/api/servidor/estado" -H "Authorization: Bearer $TOKEN" | jq .
fi
```

Resultado de la Prueba: Comprueba que el sexto intento devuelva el código 429 con el payload de bloqueo temporal. Esto demuestra que la compuerta perimetral rechaza ataques automatizados sin sobrecargar la CPU del servidor con cálculos de hash innecesarios.

12.1 CHECKLIST DE AUDITORÍA: CRITERIOS MANDATORIOS DE SEGURIDAD BACKEND

PRODUCTION QUALITY AUDIT

Matriz de Validación Técnica de Seguridad y Rendimiento

#	Criterio de Aceptación (Seguridad y Arquitectura)	Método de Verificación y Resultado Exitoso
01	Validación de Entorno Fail-Fast	El servidor se apaga inmediatamente si falta `JWT_SECRET` al arrancar.
02	Express 5 Captura Asíncrona	Excepciones en funciones `async` son capturadas sin bloquear el proceso.
03	Arquitectura en Capas Limpia	Rutas, Controladores y Servicios totalmente desacoplados en carpetas.
04	Esquemas de Entrada con Zod	Payloads inválidos retornan código 422 con array de issues descriptivos.
05	Cabeceras Ocultas con Helmet	La cabecera `X-Powered-By` está completamente ausente en `curl -I`.
06	CORS con Lista Blanca Estricta	Peticiones desde orígenes no autorizados son rechazadas de inmediato.
07	Protección Anti-DoS de Payload	Cuerpos JSON mayores a 10 kB retornan código `413 Payload Too Large`.
08	Rate Limiting Activo	5 intentos fallidos de login consecutivos activan bloqueo HTTP 429.
09	Bcrypt con Salt Dinámica	Contraseñas cifradas con `cost factor = 12`; cero texto plano en BD.
10	Timing Attack Mitigado	Comparación con DUMMY_HASH ante correos inexistentes (~140ms).
11	Access Tokens de Vida Corta	Tokens JWT expiran en máximo 15 minutos firmados con algoritmo HS256.
12	Middleware de Excepciones 4 Args	Middleware centralizado con firma `(err, req, res, next)` activo.

Técnicas de Auditoría con Herramientas de Penetración

COMANDOS DE TERMINAL PARA AUDITORÍA DE SEGURIDAD — Security Audit CLI

```
# 1. Inspeccionar todas las cabeceras HTTP emitidas por el servidor:
curl -s -I http://localhost:3000/health

# 2. Auditar vulnerabilidades conocidas en dependencias instaladas:
npm audit --audit-level=high

# 3. Probar inyección de payloads gigantescos (Protección contra DoS de memoria):
head -c 20M /dev/urandom | curl -s -X POST http://localhost:3000/api/auth/register \
  -H "Content-Type: application/json" --data-binary @- -o /dev/null -w "%{http_code}\n"
# Resultado exitoso esperado: Código HTTP 413 (Payload Too Large)
```

El Estándar de Calidad Onetel C.A.: Ninguna API backend se despliega a servidores de planta o nube si presenta vulnerabilidades altas en `npm audit` o si incumple el cifrado de contraseñas con factor de costo 12.

HOJA DE REFERENCIA RÁPIDA (CHEAT SHEET: NODE.JS & SEGURIDAD)

REFERENCIA RÁPIDA

TEXT: SNIPPETS DE PRODUCCIÓN PARA DESARROLLADORES BACKEND — Node.js Backend Quick Reference

```
# --- COMANDOS DE EJECUCIÓN NATIVOS EN NODE.JS 24 LTS ---
node --watch --env-file=.env src/server.js       # Recarga automática con .env nativo
node --test --test-reporter=spec                 # Test runner nativo sin dependencias
npm audit fix --force                            # Corrección de vulnerabilidades

# --- MIDDLEWARES DEFENSIVOS ESENCIALES ---
app.use(helmet());                               # Cabeceras de seguridad HTTP
app.use(cors({ origin: env.CORS_ORIGIN }));      # Restricción de orígenes autorizados
app.use(express.json({ limit: '10kb' }));        # Mitigación de payload flooding

# --- CRIPTOGRAFÍA CON BCRYPT ---
const hash = await bcrypt.hash(password, 12);    # Cifrado seguro con salt dinámica
const ok = await bcrypt.compare(pass, hash);     # Verificación segura

# --- TOKENS JWT (RFC 7519) ---
const token = jwt.sign(claims, SECRET, { expiresIn: '15m', algorithm: 'HS256' });
const payload = jwt.verify(token, SECRET, { algorithms: ['HS256'] });

# --- RATE LIMITING ---
app.use('/api/auth/login', rateLimit({ windowMs: 15*60*1000, max: 5 }));
```

Matriz de Códigos de Respuesta HTTP en APIs RESTful

Código HTTP	Nombre Estándar	Cuándo Emitirlo según el Estándar Onetel
200 OK	Success	Consulta, actualización o eliminación resuelta con éxito.
201 Created	Resource Created	Creación exitosa de un nuevo recurso (ej. registro de usuario o nodo).
401 Unauthorized	Unauthenticated	Falta token JWT, firma inválida o sesión expirada.
403 Forbidden	Forbidden Access	Token válido pero el usuario carece de permisos RBAC para la acción.
422 Unprocessable	Validation Error	El cuerpo JSON no cumple con el esquema estricto de Zod.
429 Too Many Req	Rate Limited	Superada la cuota máxima permitida de solicitudes por minuto/IP.